

Задание 1. Анализ масштабируемости умножения матрицы на вектор с помощью OpenMP

Описание вычислительного узла

Вычислительный узел:

- **Наименование сервера:** ProLiant XL270d Gen10
- **Процессор:** 2 × Intel Xeon Gold 6248 @ 2.50 GHz (всего 40 ядер, 80 логических потоков)
- **Архитектура:** x86_64
- **NUMA-ноды:** 2 ноды
 - Node 0: 385636 MB ($\approx$ 376.6 GiB)
 - Node 1: 387008 MB ($\approx$ 378.0 GiB)
- **Операционная система:** Ubuntu

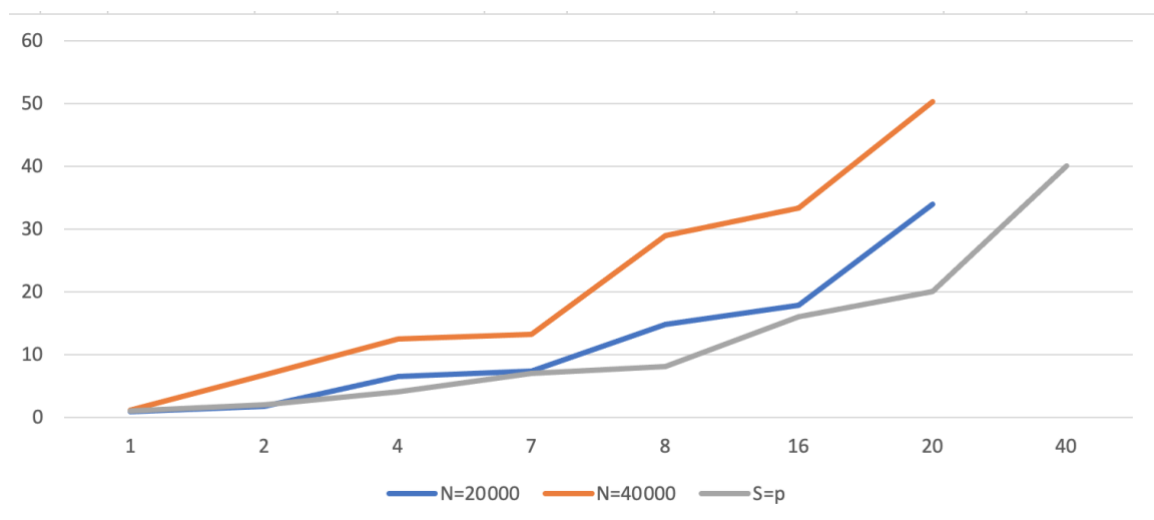
Характеристики CPU:

- Модель: Intel(R) Xeon(R) Gold 6248 CPU @ 2.50GHz
- Количество сокетов: 2
- Ядер на сокет: 20
- Потоков на ядро: 2
- Общее количество логических ядер: 80
- Максимальная частота: 3.9 GHz

Таблица ускорения (без привязки к NUMA-ноде)

M = N	1	2	4	7	8	16	20	40
20 000 (~3 GiB)	0.5824	0.6884	0.3398	0.0896	0.0792	0.0396	0.0327	0.0171
S_p	1.00	0.85	1.71	6.50	7.36	14.72	17.81	33.96
40 000 (~12 GiB)	5.8754	5.0476	0.8800	0.4741	0.4454	0.2037	0.1766	0.1170
S_p	1.00	1.16	6.68	12.39	13.19	28.85	33.28	50.22

График: Зависимость ускорения от количества потоков



Вывод по Заданию 1

При запуске без привязки к NUMA-ноде программа демонстрирует **сверхлинейное ускорение**. Для матрицы $20\,000 \times 20\,000$ ускорение достигает **33.96×** на 40 потоках. Для матрицы $40\,000 \times 40\,000$ ускорение ещё выше — **50.22×**.

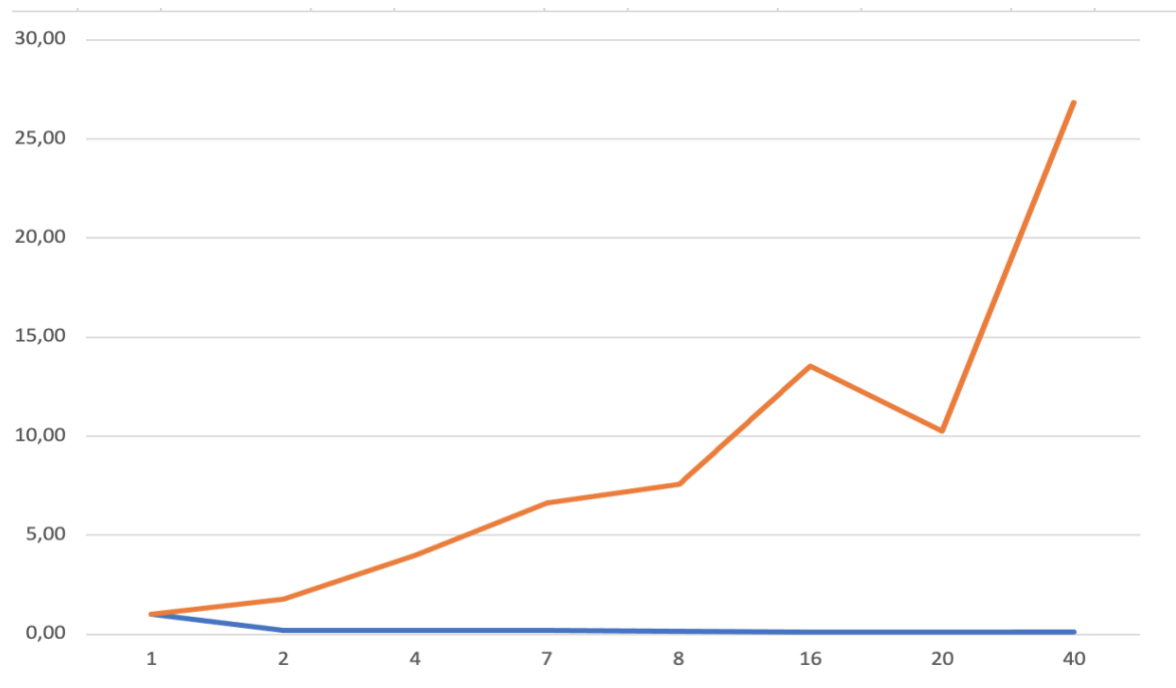
Такой результат объясняется эффективным распределением потоков по двум NUMA-нодам сервера, что позволяет лучше использовать суммарный кэш и пропускную способность памяти. При увеличении размера матрицы масштабируемость улучшается.

Задание 2. Параллельное численное интегрирование

Сравнение двух реализаций

Потоков	Atomic Time (c)	Atomic S_p	Local Time (c)	Local S_p	Local быстрее Atomic
1	0.4293	1.00	0.04528	1.00	9.48×
2	2.295	0.19	0.02561	1.77	89.61×
4	2.429	0.18	0.01132	4.00	214.62×
7	2.341	0.18	0.00683	6.63	342.84×
8	2.659	0.16	0.00599	7.56	443.98×
16	3.523	0.12	0.00334	13.54	1053.19×
20	3.558	0.12	0.00441	10.26	806.35×
40	3.923	0.11	0.00169	26.82	2323.72×

График: Зависимость ускорения от количества потоков



оранжевый – Atomic, синий - Local

Вывод по Заданию 2

Были реализованы две версии функции численного интегрирования:

- С использованием `#pragma omp atomic`
- С использованием локальной переменной в каждом потоке

Версия с `atomic` показала **крайне плохую масштабируемость** — время выполнения значительно возрастает с ростом числа потоков из-за сильной конкуренции за общую переменную.

Версия с **локальной переменной** продемонстрировала **хорошую масштабируемость**, достигнув ускорения **26.82 раза** на 40 потоках. Данный подход оказался в тысячи раз эффективнее `atomic`-варианта при большом количестве потоков.